CHAPTER 1

DESIGNING ATM

1 Getting Ready: The ATM System 2
2 Requirements for the ATM System 13
3 Use Case Diagram for the ATM System 14
4 Class Diagram for the ATM System 17
5 Sequence Diagram for the ATM System 28
6 Activity Diagram for the ATM System 29
7 Code for the ATM System . 32
8 Summary: Designing ATM . 40
9 Key Insights and Tips: Designing ATM 44

In this chapter, you will discover the design of an ATM system, focusing on requirements, use case, class, sequence, activity diagrams, and implementation code.

1 Getting Ready: The ATM System

1.1 Problem definition

An **automated teller machine (ATM)** is a self-service terminal that enables bank customers to perform financial transactions without interacting with a human teller or visiting a physical bank branch. Through ATMs, users can perform essential banking operations such as depositing cash, withdrawing money, checking account balances, and transferring funds between accounts. ATMs are typically placed in accessible public locations like banks, malls, airports, and convenience stores to provide round-the-clock banking services.
In this LLD interview case study, you'll focus on:

- Enabling secure card-based access and PIN verification for users.
- Supporting core transactions: cash withdrawal, deposit, balance inquiry, and fund transfer.
- Managing cash inventory within the ATM, including withdrawal limits and handling insufficient funds.
- Ensuring a seamless and secure user experience through well-coordinated hardware and software components.

Note: The design can be adapted for modern ATMs with features like contactless cards, mobile integrations, or advanced security (e.g., biometrics), but we will focus on the core ATM functionalities.

1.2 Expectations from the interviewee

During the interview, you are expected to discuss key aspects of the ATM system's design, clarifying requirements, handling edge cases, and reasoning about hardware-software interactions. Typical areas to cover include:

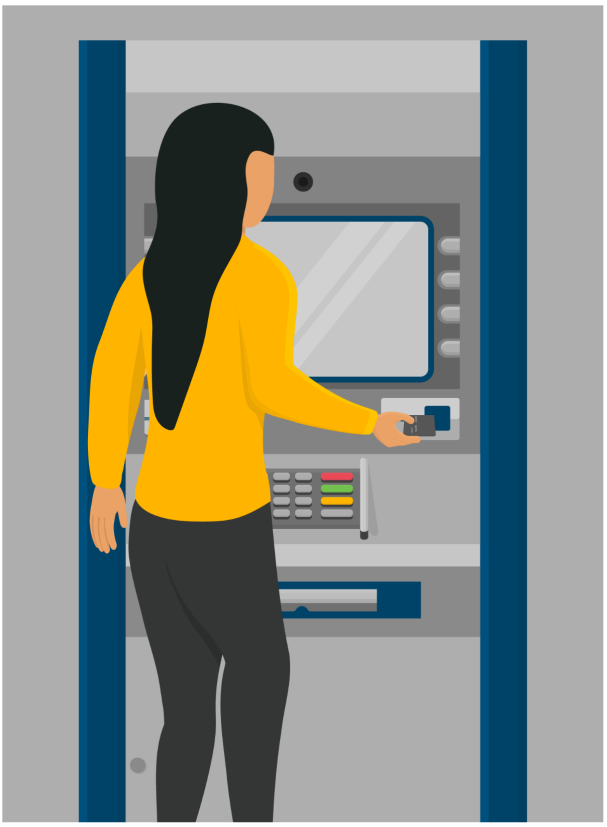
ATM components

To better understand an ATM system, you may ask the interviewer the following questions:

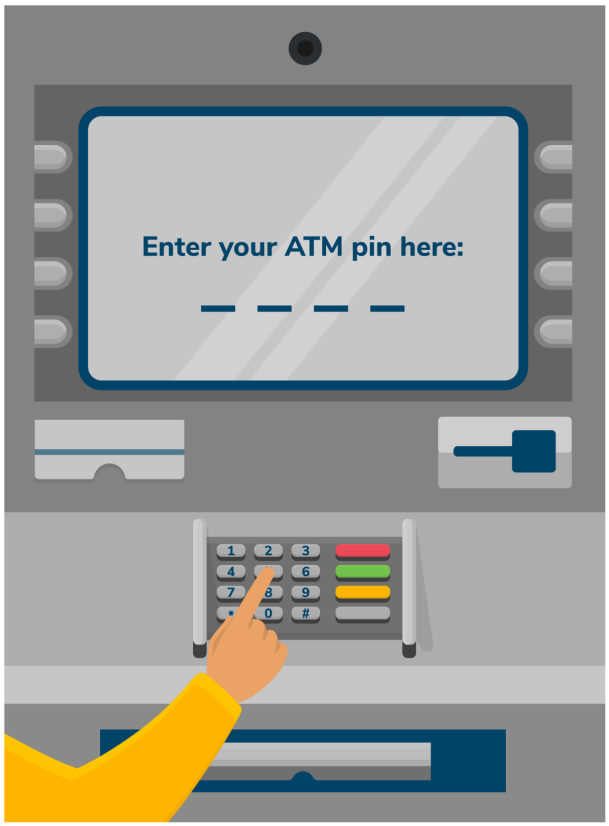
1. What are the components of an ATM?
2. Is the ATM necessarily placed inside a room?
3. Does an ATM have a fingerprint scanner?

ATM features

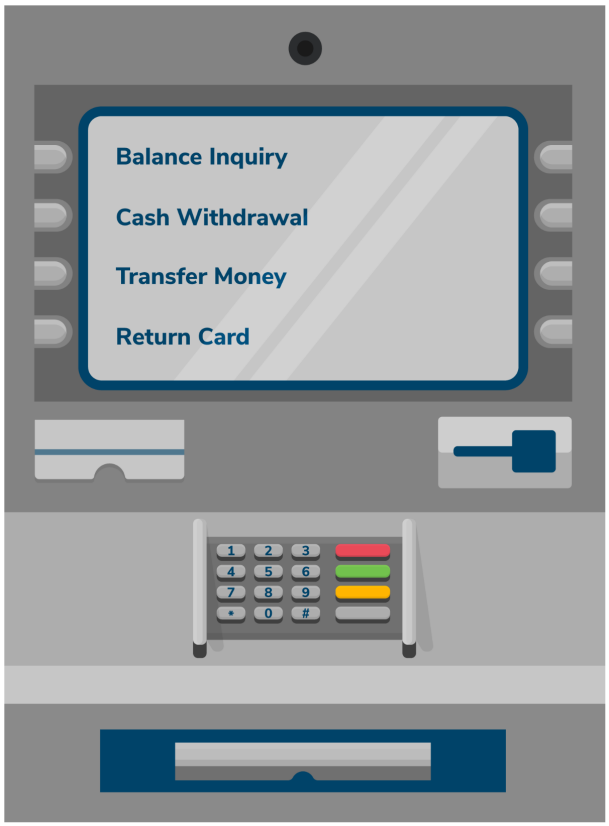
Different ATMs may vary in terms of features, which is why it is important to clear the following questions from the interviewer:



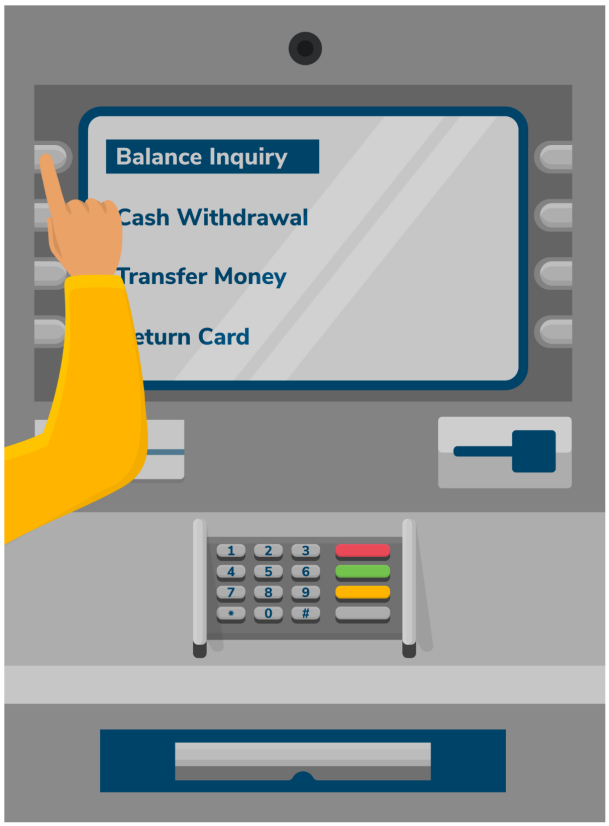
(a) The user inserts their card in the ATM



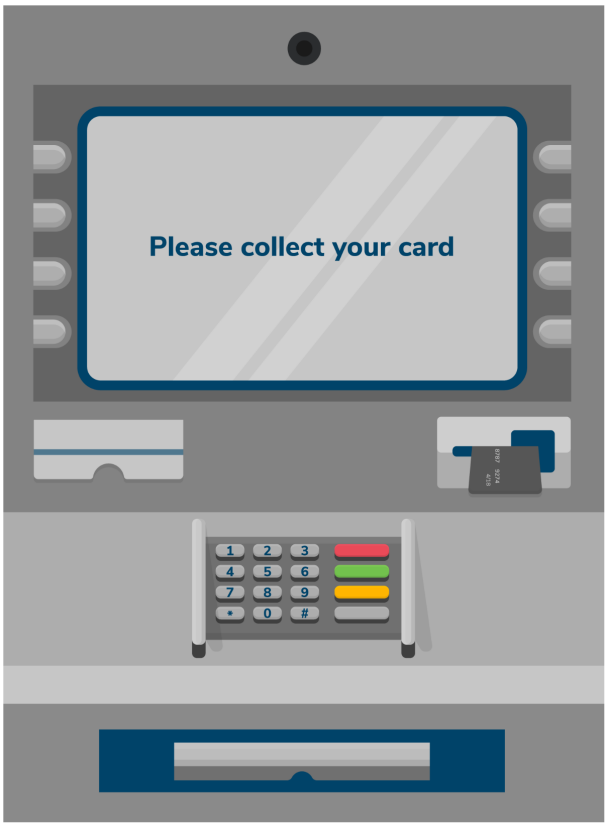
(b) The ATM reads the information on the card and prompts the user to enter their PIN



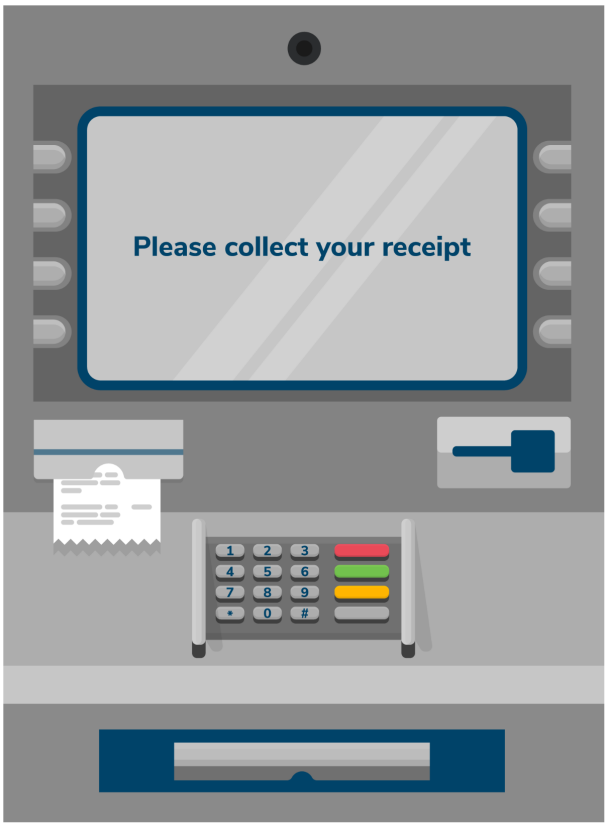
(a) The options above appear on the ATM upon successful PIN authentication



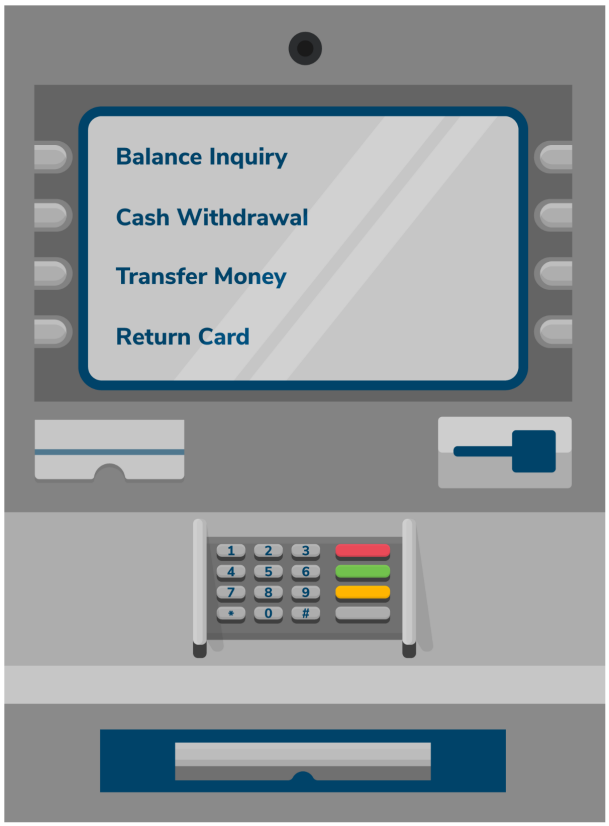
(b) The user selects the "Balance Inquiry" option



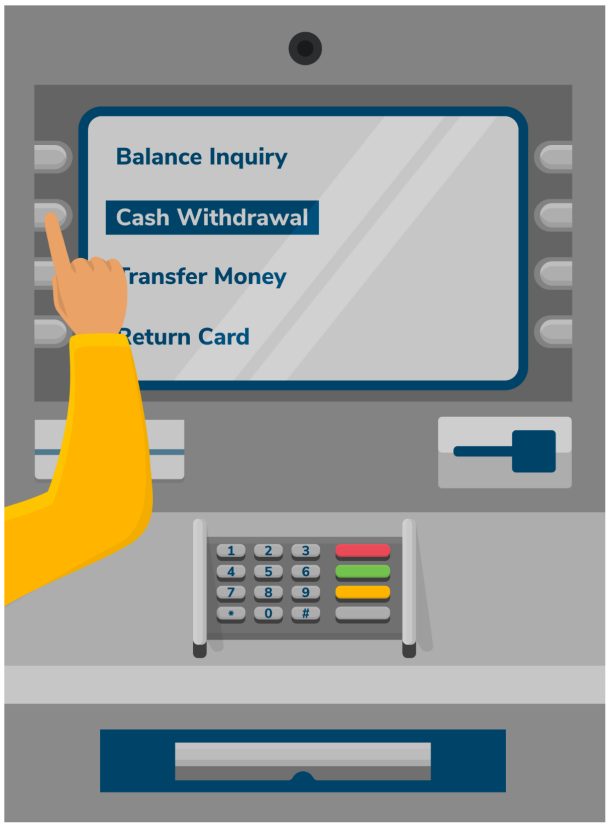
(a) The ATM asks the user to collect the card



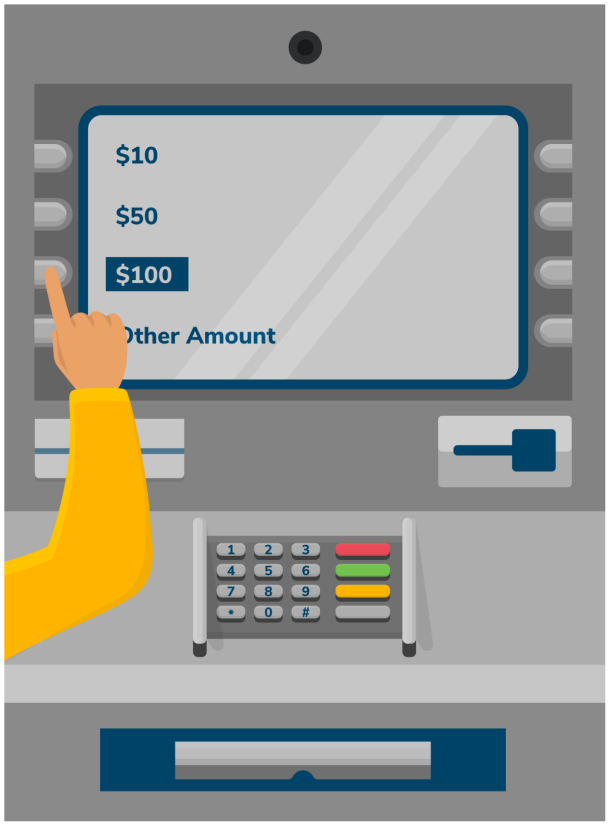
(b) The ATM asks the user to collect the balance receipt



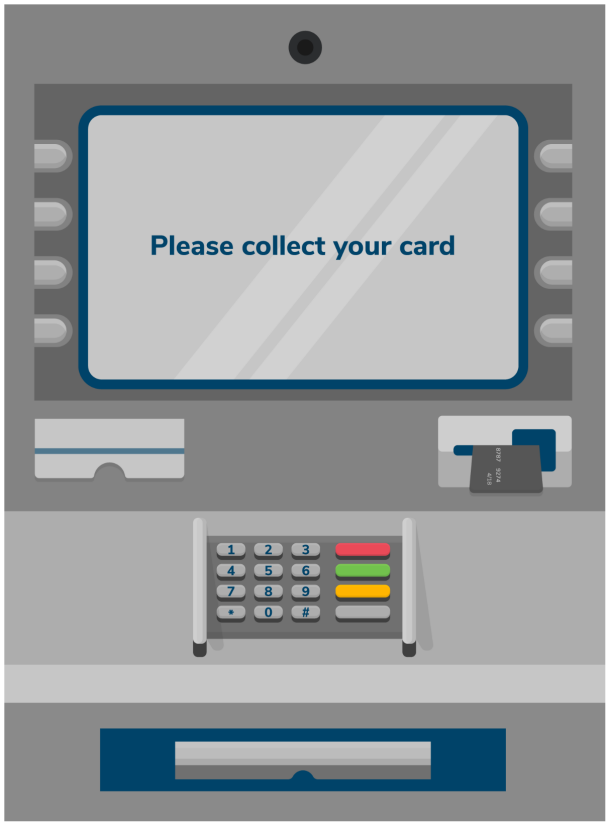
(a) The options above appear on the ATM upon successful PIN authentication



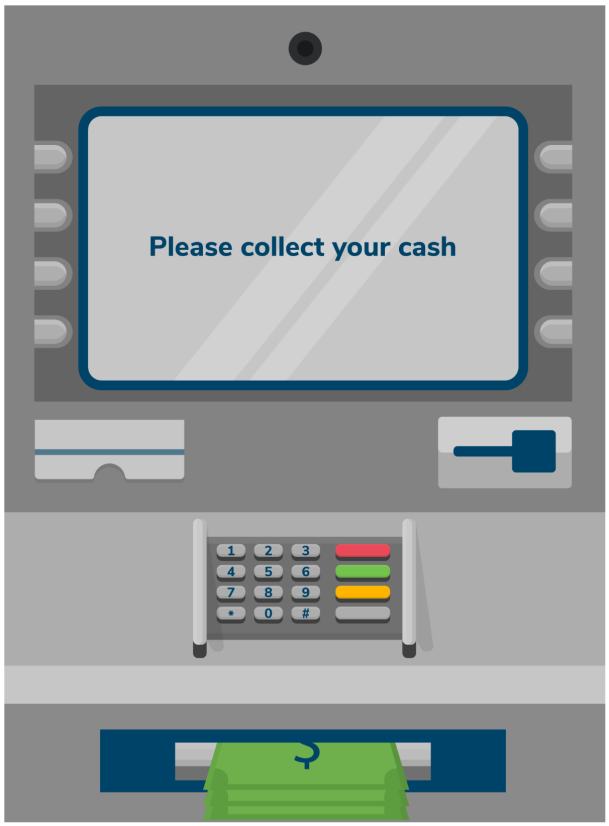
(b) The user selects the "Cash Withdrawal" option



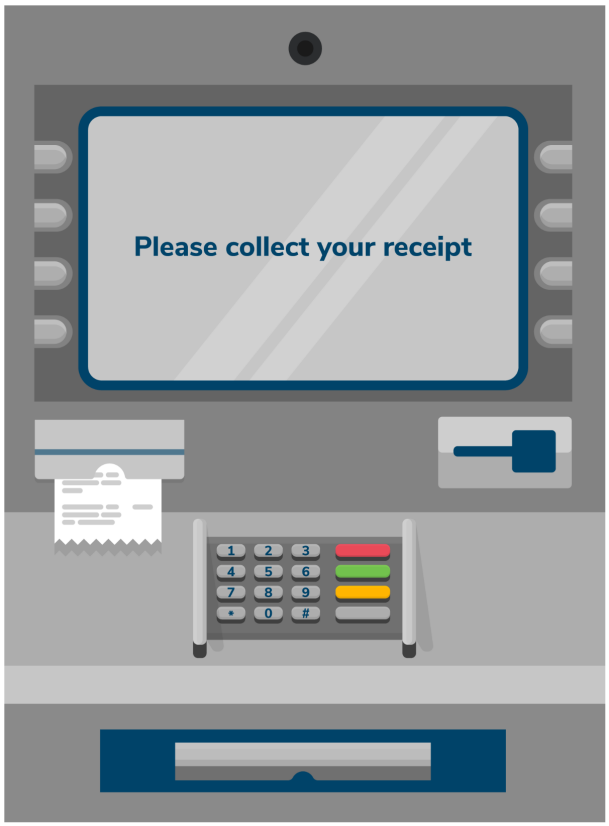
(a) The user selects the "$100" option



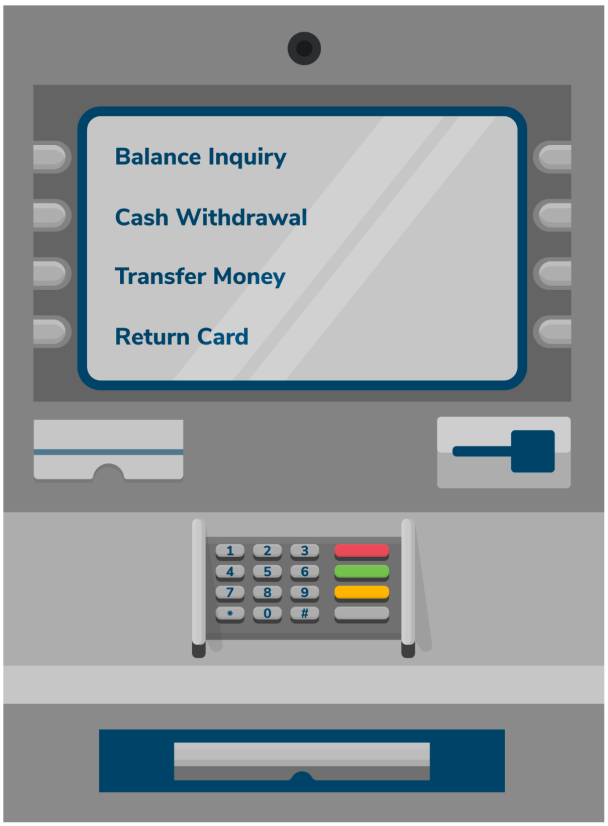
(b) The ATM asks the user to collect the card



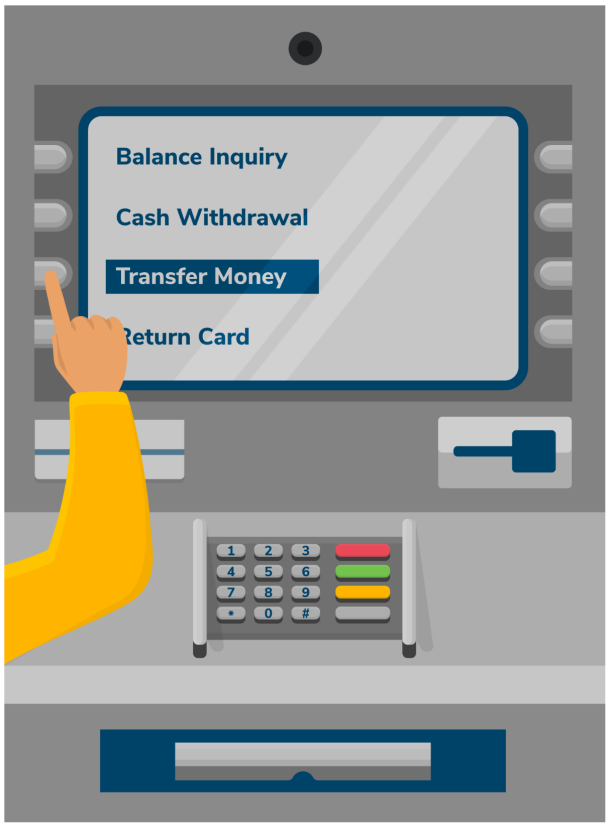
(a) The ATM asks the user to collect the cash according to the amount specified



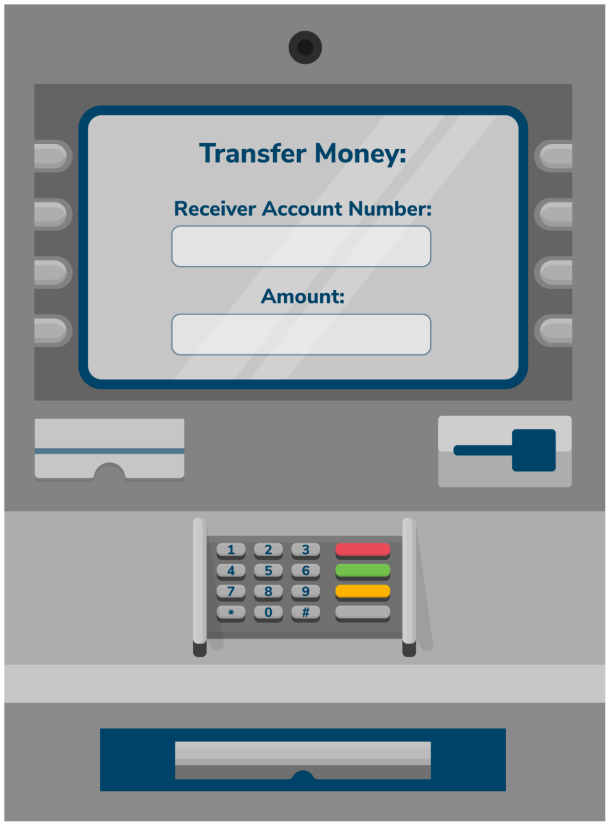
(b) The ATM asks the user to collect the receipt



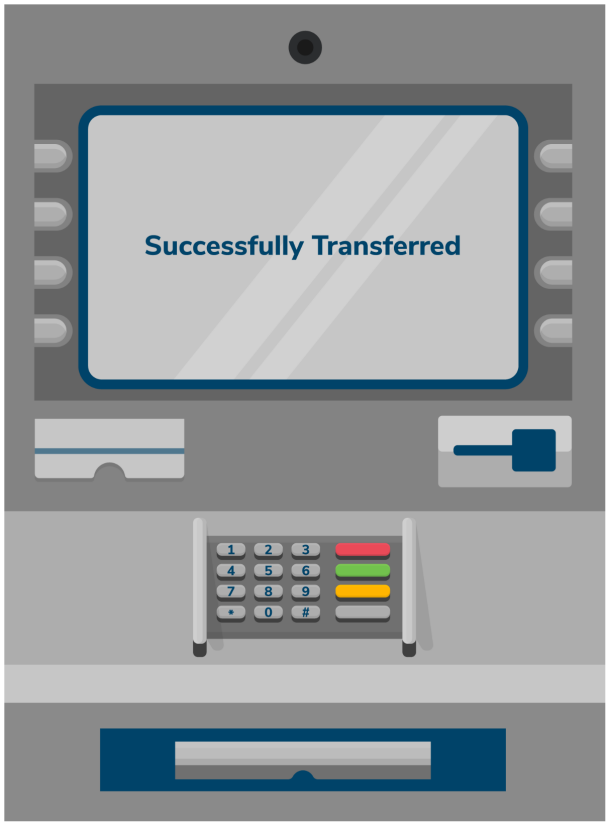
(a) The options that appear on the ATM upon successful PIN authentication



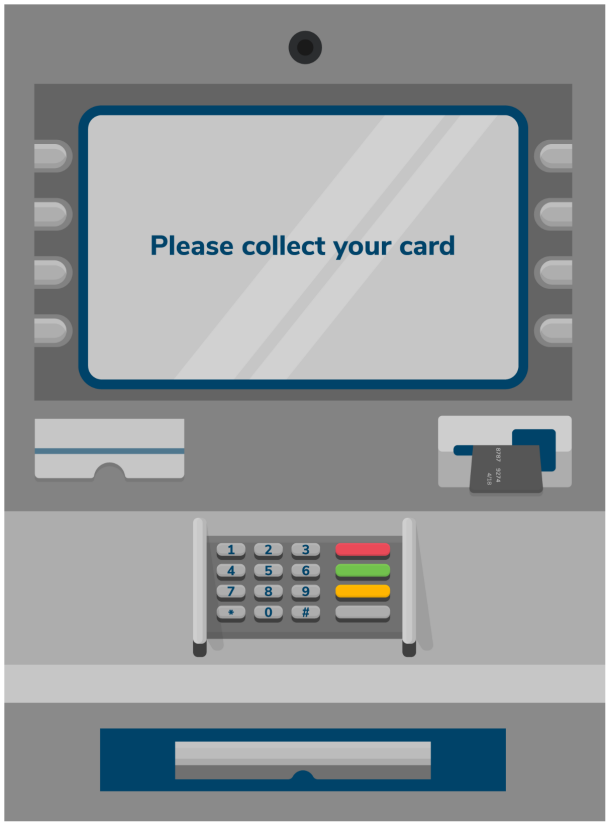
(b) The user selects the "Transfer Money" option



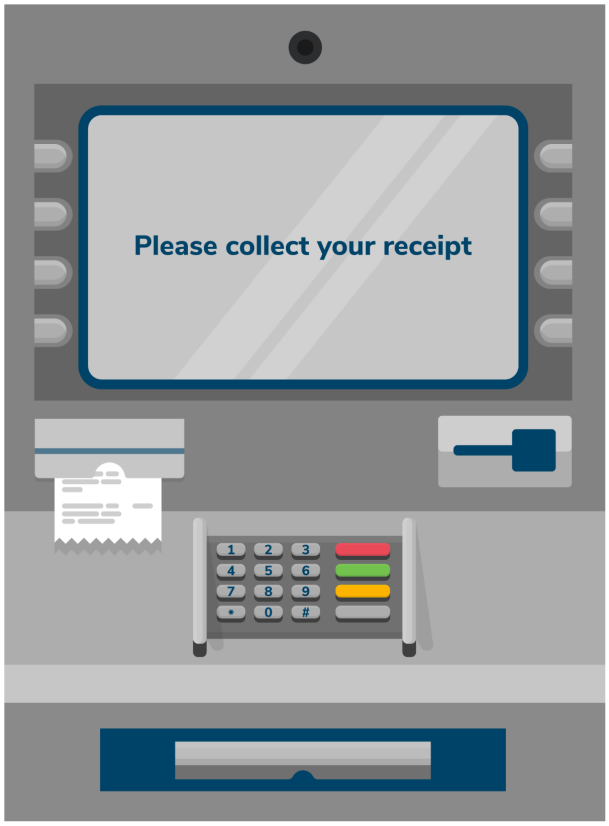
(a) The ATM prompts the user to enter the receiver account number and the amount to be sent



(b) The ATM notifies the user that the transaction was successful



(a) The ATM asks the user to collect the card



(b) The ATM asks the user to collect the receipt

1. What is the withdrawal limit of an ATM?
2. Can we check our account balance using an ATM?
3. Can we set a PIN using an ATM?

ATM processing

The interviewer would expect you to ask a question about processing ATM transactions. Therefore, you may ask the following questions:

1. What happens when the amount the user enters for withdrawal exceeds the user's account balance?
2. What happens when the amount the user enters for withdrawal exceeds the ATM's cash limit?
3. What happens when the amount the user enters exceeds the total cash in the ATM?
4. Can the ATM be used for online transactions?

1.3 Design approach

We will design this ATM system using a *bottom-up approach*, following these steps:

- First, we'll identify and model the core hardware components such as `CardReader`, `Keypad`, `Screen`, `CashDispenser`, and `Printer`, defining their responsibilities and interactions.
- Next, we'll compose these components into larger entities representing the ATM's operational state, transaction workflows, and the overall machine.
- We'll model how the ATM interacts with users for authentication, processes various transactions, manages its cash inventory, and ensures secure and reliable operations.
- This approach will allow us to address edge cases, hardware failures, and incorporate SOLID design principles for modularity and maintainability. We 'll use diagrams and code examples to illustrate the design at each step.

1.4 Design pattern

During an interview, discussing the design patterns under which the ATM system falls is always a good practice. Stating the design patterns gives the interviewer a positive impression and shows that the interviewee is well-versed in the advanced concepts of object-oriented design.
Try to answer the following question. If you are not familiar with design patterns, don't worry! You can learn about them by asking, "Define design patterns."

Question:

Which design pattern(s) should be used to design an ATM system? Please elaborate on your choice(s).

Let's explore the requirements of the ATM system in the next lesson.

1.5

2 Requirements for the ATM System

In this lesson, we outline the functional and operational requirements for the ATM System. Identifying and understanding requirements is essential to defining the system's scope and ensuring a robust, user-friendly design.
We'll use the notational convention to identify each requirement with a unique label, "Rn," where "R" is short for requirement, and "n" is a natural number.

2.1 Requirement collection

- **R1:** Each user has a single account at the bank that they can access by inserting their card into the ATM.
- **R2:** The ATM system consists of the following main components to facilitate user interaction and transaction processing:
  - **Card Reader:** Reads the user's ATM card.
  - **Keypad:** The user can enter their PIN and transaction details.
  - **Screen:** Displays prompts, messages, and transaction information.
  - **Cash Dispenser:** Dispenses cash to the user for withdrawal transactions.
  - **Printer:** Prints transaction receipts for the user.
  - **Network Infrastructure:** Securely connects the ATM to the bank's central system to process transactions and access account information.
- **R3:** The ATM must authenticate users by verifying the PIN entered, ensuring that only authorized customers can access account services.
- **R4:** All transaction options become available only after successfully authentication the card and PIN.
- **R5:** Users may have either or both of the following account types:
  - Current account
  - Savings account

- **R6:** The ATM must allow the following operations for each account type:
  - Balance inquiry
  - Cash withdrawal
  - Funds transfer
- **R7:** After completing a transaction, the user should have the option to perform another transaction or end their session and retrieve their card.

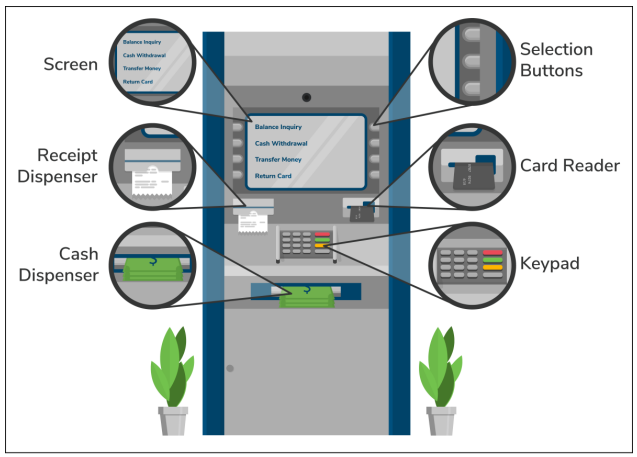


Figure 1.10: ATM components

We've identified our requirements for the problem. In the next lesson, let's define different use cases of the ATM system.

3 Use Case Diagram for the ATM System

Let's build the ATM system's use case diagram and understand the relationship between its main actors and functions. First , we'll define the different elements of our system, followed by the complete use case diagram.

3.1 System

Our system is the ATM system. It manages secure, card-based access for bank customers to perform key banking transactions remotely.

3.2 Actors

Let's define the main actors of our ATM system.

Primary actor

- **Cardholder:** The user who interacts with the ATM to perform banking transactions such as inserting/removing the card, entering PIN, and carrying out account operations.

3.3 Use cases

In this section, we will define the ATM's use cases. We have listed them according to their interactions with a particular actor.

Cardholder

- **Insert card:** To insert the ATM card into the machine.
- **Enter PIN:** To enter their PIN to verify identity.
- **Change PIN:** To change the card's PIN.
- **Select transaction:** To initiate a transaction (generalization of the next four):
  - **Balance inquiry:** To check the account balance.
  - **Funds transfer:** To transfer money between accounts.
  - **Cash withdrawal:** To withdraw the cash.
- **Enter amount:** To enter the amount they want to transfer or withdraw.
- **Cancel transaction:** To cancel a transaction.

ATM system

- **Verify cardholder identity:** To validate the card and the cardholder's credentials.
- **Check withdrawal limits:** To validate the ATM's transaction limits and the cardholder's bank limits.
- **Check account transaction limits:** To validate that the transaction is within the account's permissible limits.
- **Return card:** To eject the card after the transaction or cancellation.
- **Dispense money:** To dispense cash during withdrawal.
- **Diispense receipt:** To print and provide a receipt for the transaction.

3.4 Relationships

This section describes the relationships between and among actors and their use cases.

Associations

The table below shows the association relationship between actors and their use cases.

Cardholder	**ATM System**
Insert card	Verify cardholder identity
Enter PIN	Check withdrawal limits
Change PIN	Check account transaction limits
Select transaction	Return card
Enter amount	Dispense money
Cancel transaction	Dispense receipt

Include

When cardholders initiate an ATM session, they must verify their identity by entering their PIN. This process is enforced using "include" relationships in the use case diagram:

- The "Insert card" use case includes the "Enter PIN" use case.
- The "Enter PIN" use case includes the "Verify cardholder identity" use case.

Before a cardholder can change their PIN, the system must authenticate them:

- The "Change PIN" use case includes the "Enter PIN" use case and then the "Verify cardholder identity" use case.

When a cardholder chooses to perform a transaction, they must select from the available transaction types:

- The "Select transaction" use case includes the "Balance inquiry", "Funds transfer", and "Cash withdrawal" use cases.

For "Funds transfer" and "Cash withdrawal", the cardholder must specify the amount to transfer or withdraw:

- The "Funds transfer" and "Cash withdrawal" use cases include the "Enter amount" use case.

When the cardholder enters an amount to withdraw, the system must ensure all limits are respected:

- The "Enter amount" use case includes the "Check withdrawal limits" use case.
- The "Check withdrawal limits" use case includes the "Check account transaction limits" use case.

After a transaction (such as a balance inquiry, funds transfer, or cash withdrawal), the system provides a receipt to the cardholder if requested:

- The "Balance inquiry" and "Cash withdrawal" use cases each include the "Dispense receipt" use case.

If the cardholder cancels a transaction or ends their session, the ATM must return the card:

- The "Cancel transaction" use case includes the "Return card" use case.
- At the end of any session, the "Return card" use case is always included to ensure the cardholder retrieves their card.

3.5 Use case diagram

Here's the use case diagram of the ATM design:

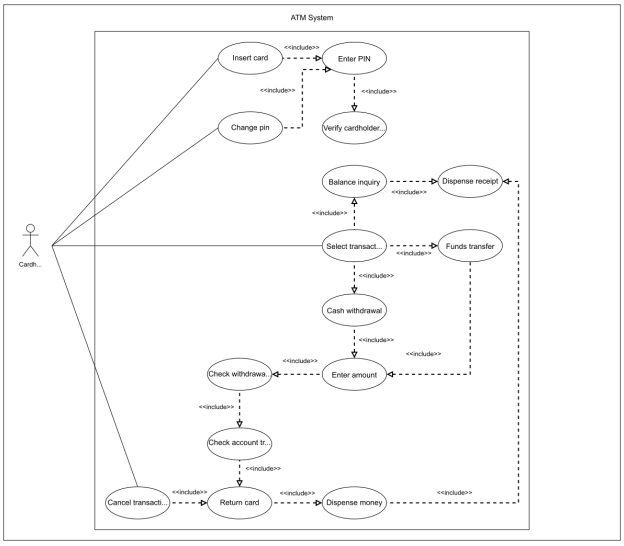


Figure 1.11: The use case diagram of the ATM system

In the next lesson, we will discuss the class diagram and explain all classes and their relationships.

4 Class Diagram for the ATM System

In this lesson, we'll design the classes and then identify the relationship between classes according to the requirements for the ATM design problem.

4.1 Components of the ATM system

As mentioned earlier, we'll design the class diagram for the ATM using a bottom-up approach.

User

The`User` class models a bank customer who uses the ATM. It holds the user's `ATMCard` and associated BankAccount and initiates ATM sessions and transactions.

Figure 1.12: The User class

R1: Each user has a single account at the bank that they can access by inserting their card into the ATM.

ATM card

The `ATMCard` class encapsulates the details required for card-based authentication and authorization. It uniquely identifies a user's bank card for ATM operations and secures access through PIN verification.

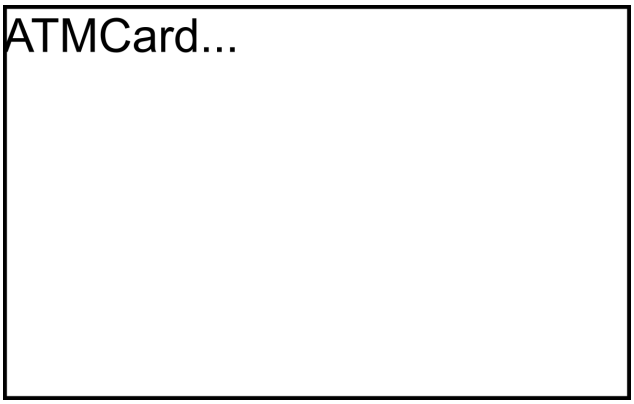


Figure 1.13: The ATMCard class

R4: All transactions are possible after the successful authentication of the ATM card.

Bank account

The `BankAccount` class represents a user's bank account. Two concrete types extend it: `SavingAccount` and `CurrentAccount`.

- `SavingAccount`**:** This derived class represents a savings account with a withdrawal limit.

- `CurrentAccount:` This derived class represents a current/checking account with a withdrawal limit.

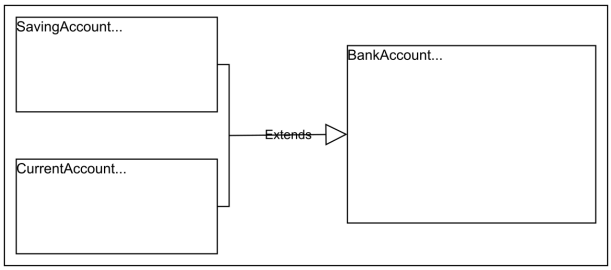


Figure 1.14: BankAccount and its derived classes

R1: Each user has a single account at the bank that they can access by inserting their card into the ATM.
R5: The user can have two types of accounts—current and savings—and can perform the following operations on the ATM: - Balance inquiry - Cash withdrawal - Funds/money transfer

Bank

The `Bank` class models a financial institution responsible for accounts and cards. It provides account validation and backend transaction processing, and associates the bank code and name with issued cards and accounts.

Figure 1.15: The Bank class

R1: Each user has a single account at the bank that they can access by inserting their card into the ATM.

Card reader, cash dispenser, keypad, screen, and printer

- `CardReader:` This class accepts or rejects a card.

- `CashDispenser`: This class provides the required amount specified by the user in cash.
- `Keypad`: This class allows the user to enter the PIN.
- `Screen`: This class represents a screen that displays information upon card insertion.
- `Printer`: This class represents a printer that prints the transaction/withdrawal receipts for the user.

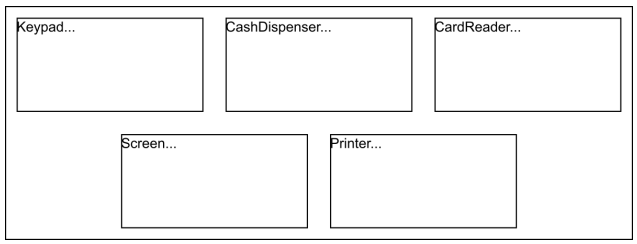


Figure 1.16: The class diagram of the Keypad, CashDispenser, CardReader, Screen, and Printer classes

R2: The main components of the ATM system that facilitate interactions between the user and the machine are listed below: - **Card reader:** To read the user's ATM card - **Keypad:** To enter information such as the user's PIN - **Screen:** To display messages to the user, such as prompts or error messages - **Cash dispenser:** To dispense cash to the user - **Printer:** To print receipts for the user - **Network infrastructure:** To connect with the bank's computer system to access account information and complete transactions

ATM state

The `ATMState` abstract class and its concrete subclasses implement the State Design pattern for the ATM. Each state handles only those operations valid during the session, enforcing a robust and safe workflow.

- `IdleState`: ATM is waiting for a card; handles card insertion.
- `HasCardState`: Card is inserted. Handles PIN entry/authentication and card return.
- `SelectionOptionState`: User can select an operation (balance inquiry, withdrawal, transfer, change PIN, or cancel).
- `BalanceInquiryState`: Handles the balance inquiry process and returns the user to operation selection.
- `CashWithdrawalState`: Handles withdrawal, including cash dispensing and updating balances.
- `TransferMoneyState`: Handles funds transfers, including validation and updating accounts.
- `ChangePinState`: Handles the process of updating a card's PIN.

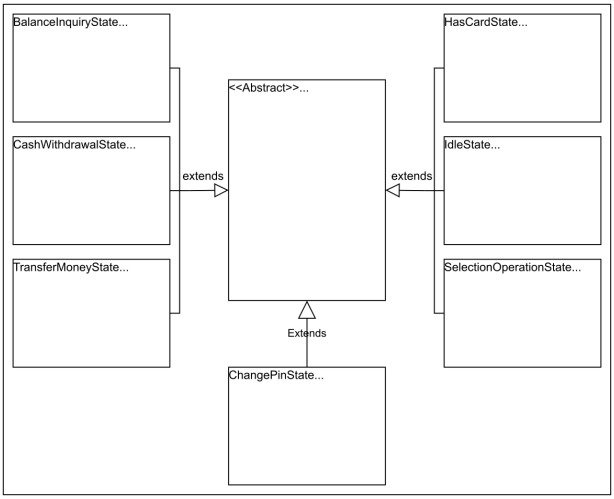


Figure 1.17: ATMState and its derived classes

ATM

The `ATM` class models the full automated teller machine, orchestrating every operation. It maintains the current state of operation (`Idle`, `HasCard` , `SelectionOption` transaction states). It also tracks current cash reserves and the count of available bill denominations (hundreds, fifties, tens).

ATM room

An `ATMRoom` class has an ATM and may or may not have a user.

Enumerations and custom data types

The following overviews the enumerations and custom data types used in this problem.

- `ATMStatus`: This enumeration keeps track of the following states of an ATM:
  - `Idle`
  - `HasCard`
  - `OptionSelected`
  - `CashWithdrawal`
  - `MoneyTransfer`
  - `DisplayBalance`
  - `ChangePIN`
- `TransactionType`: This enumeration represents the following transactions:
  - `BalanceInquiry`

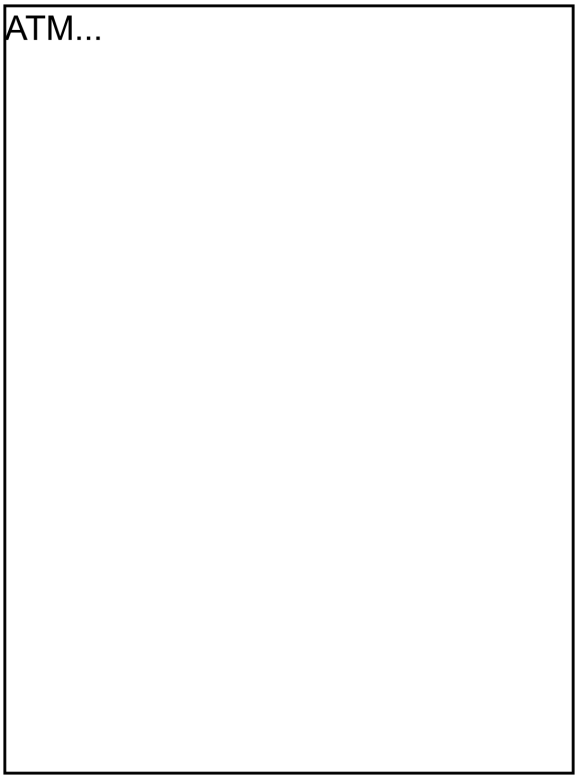


Figure 1.18: The ATM class

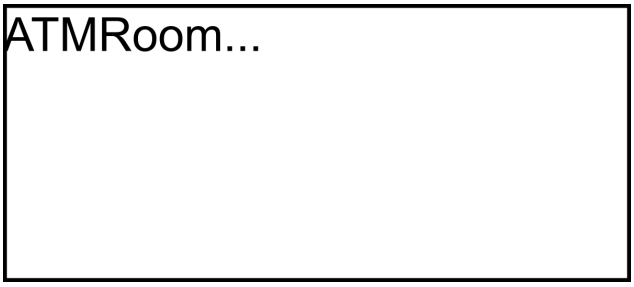


Figure 1.19: The ATMRoom class

  - `CashWithdrawal`
  - `FundsTransfer`
  - `ChangePIN`
  - `Cancel`

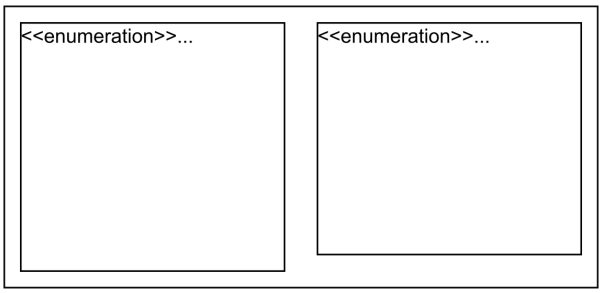


Figure 1.20: Enums in the ATM design problem

4.2 Relationship between the classes

Association

The class diagram has the following association relationships:

- `User` is associated with (has a reference to) these objects but does not "own" their life cycle (they can exist without the user object in code).
  - `ATMCard` (one-to-one)
  - `BankAccount` (one-to-one)
- `ATM` is associated with:
  - `Bank` (typically for backend operations or validation; not always a direct attribute in your code, but conceptually, for real transaction validation)
  - `ATMState` (the current session state; owned but could be swapped, so this is a weak composition or aggregation)
  - `User` (the current active user during a session; reference, not strong ownership)
  - `ATMCard` (the currently inserted card in the session)
- `ATMCard` is associated with (via the user) a `BankAccount` (conceptually, not as a direct field).

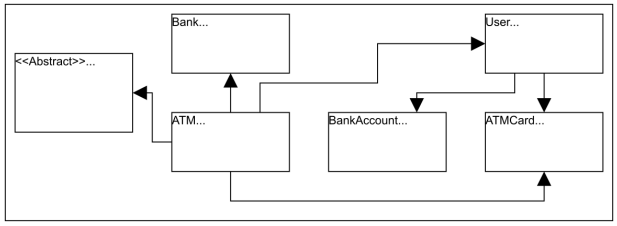


Figure 1.21: The association relationship between classes

Composition

The class diagram has the following composition relationships.

- `ATM` is composed of the following hardware components:
  - `CardReader`
  - `CashDispenser`
  - `Keypad`
  - `Screen`
  - `Printer`
- These are usually instantiated and owned by the `ATM` object. If the `ATM` is destroyed, these components cease to exist as well. This is a strong (composition) relationship.

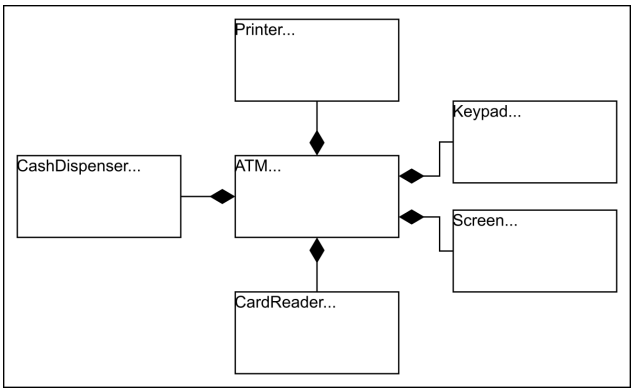


Figure 1.22: The composition relationship between classes

Inheritance

The following classes show an inheritance relationship:

- `SavingAccount` and `CurrentAccount` both extend the abstract class `BankAccount`.

- All concrete state classes (IdleState, HasCardState, SelectionOptionState, BalanceInquiryState, CashWithdrawalState, TransferMoneyState, ChangePinState) extend the abstract class ATMState.

  Note: We have already discussed the inheritance relationship between classes in the component section above one by one.

4.3 Class diagram for the ATM System

In this section, we outline the multiplicity (cardinality) relationships between the main classes in our ATM system. We explain the allowed number of instances on each side for each relationship and the real-world or design rationale behind the connection. Understanding these relationships is key to modeling how different entities interact and collaborate to support key workflows in the system.

Source	**Target**	**Multiplicity**	**Reason**
ATM	CardReader	1--1	Every ATM has exactly one `CardReader` (composition).
ATM	CashDispenser	1--1	Every ATM has exactly one `CashDispenser` (composition).
ATM	Keypad	1--1	Every ATM has exactly one `Keypad` (composition).
ATM	Screen	1--1	Every ATM has exactly one `Screen` (composition).
ATM	Printer	1--1	Every ATM has exactly one `Printer` (composition).
ATM	ATMState	1--1	ATM holds a reference to exactly one current state at any time.
ATM	User	1--0..1	ATM may have one active user during a session, or none if idle.
ATM	ATMCard	1--0..1	ATM may have one inserted card during a session, or none if idle.

ATM	Bank	1--1	ATM belongs to/operates with one bank (assumed in design).
User	ATMCard	1--1	Each `User` is issued exactly one `ATMCard`.
User	BankAccount	1--1	Each `User` has exactly one `BankAccount`.

Here's the complete class diagram for our ATM design:

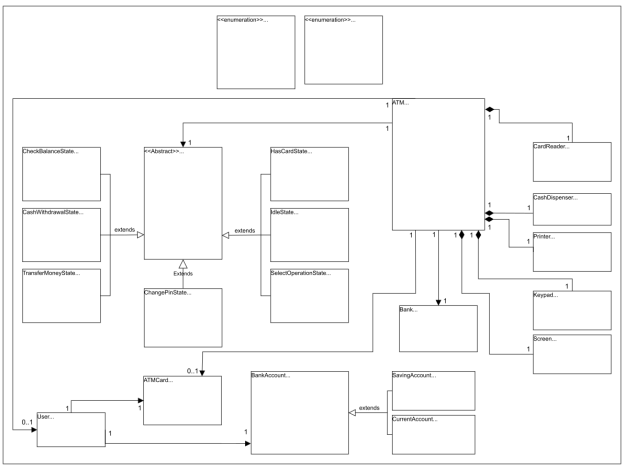


Figure 1.23: The class diagram of the ATM design problem

4.4 Design pattern

The following design patterns have been used in the class diagram:

- **The Singleton design pattern:** This pattern ensures the existence of a single instance of the ATM at a given moment that can be accessed by multiple users, due to the shared nature of the ATM components.
- **The State design pattern:** This pattern enables the ATM to alter its behavior based on internal changes in the machine. For example, an ATM can transition from one state to another, like switching from an idle state to displaying an account balance or money withdrawal state, and as soon as all the operations have been performed, it can switch back to the initial idle state.

The following design patterns can also be used to design an ATM:

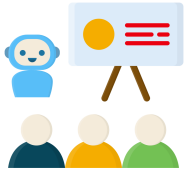
- The Composite design pattern can combine different ATM components and functionalities.

- The Builder design pattern allows the same processes for a complex object to have different representations. In the ATM system, it can help separate different kinds of transactions, such as withdrawals and deposits.

We have completed the class diagram of the ATM system according to the requirements. In the next lesson, let's design its sequence diagram.

4.5 AI-powered trainer

At this stage, everything should be clear. If you encounter any confusion or ambiguity, feel free to utilize the interactive AI-enabled widget below to seek clarification. This tool is designed to help you strengthen your understanding of the concepts.



4.6 Additional requirements

The interviewer might ask about the workings of the cash withdrawal process. How can it be implemented in our ATM system? This addition is a bit challenging since we need a system that can withdraw the correct combinations of hundred, twenty, and two-dollar bills, respectively, according to the amount specified by the user. The system also needs to work sequentially until the required amount is met.
We will use the *Chain of Responsibility* design pattern to tackle this addition to our system. This design pattern will ensure the correct division of the dollar bills in the ATM by creating a chain of handlers that forward the requests based on the situation until all the requirements are met.
We have created the following classes to implement the Chain of Responsibility design pattern:

- CashWithdrawProcessor: Associated with the CashWithdrawalState class. This abstract class is extended by HundredDollarWithdrawProcessor, FiftyDollarWithdrawProcessor, and TwoDollarWithdrawProcessor.

- HundredDollarWithdrawProcessor: Derived from CashWithdrawProcessor and responsible for dispensing hundred-dollar bills.

- FiftyDollarWithdrawProcessor: Derived from CashWithdrawProcessor and responsible for dispensing fifty-dollar bills.

- TwoDollarWithdrawProcessor: Derived from CashWithdrawProcessor and responsible for dispensing two-dollar bills.

  Valid amount: If the amount entered by the user has a modulus equal to zero with any specified bills that the ATM can withdraw, then the amount is considered valid for the transaction. If the amount is invalid, then the transaction will not be processed.

For example, a user wants to withdraw $548. The `HundredDollarWithdrawProcessor` class will start the cash withdrawal using the `cashWithdrawal()` method by taking out five bills of one hundred dollars. Now that we have $48 to withdraw for the user, less than a hundred dollars, the `FiftyDollarWithdrawProcessor` class will start withdrawing dollar bills. This class will take out two bills of twenty dollars with $8 remaining. Since two dollars is less than twenty, the `cashWithdrawal()` method of the `TwoDollarWithdrawProcessor` will take out four bills of $2 for the user. The withdrawal, in this case, is successful.

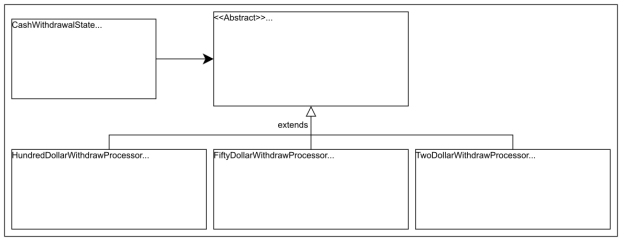


Figure 1.24: How to apply the Chain of Responsibility design pattern to the cashWithdrawal method

We have completed the class diagram of the ATM system according to the requirements. In the next lesson, let's design its sequence diagram.

5 Sequence Diagram for the ATM System

A sequence diagram is a great way to understand the interactions between different entities and objects in the system. We can create different sequence diagrams for our ATM. For the sake of this lesson, we will create sequence diagrams for balance inquiry from the ATM.

5.1 Balance inquiry

The sequence diagram for how to complete the balance inquiry using an ATM should have the following actors and objects that will interact with each other:

Actors and Objects

- **Actor:** User (the person using the ATM)

- **Objects:** ATM, Card issuer (Bank), and Printer

Steps for balance inquiry using an ATM:

1. The user inserts their ATM card into the ATM.
2. The user enters their PIN on the ATM.
   (a) The ATM receives the entered PIN and sends a verification request to the card issuer (Bank).
3. The card issuer (Bank) verifies the PIN:
   (a) If the PIN is valid, the Bank notifies the ATM that verification was successful.
   (b) If the PIN is invalid, the Bank notifies the ATM, which returns the card to the user, ending the session.
4. If the PIN is verified:
   (a) The user selects the "Balance Inquiry" operation on the ATM.
   (b) The ATM requests the card issuer (Bank) to retrieve the account balance.
   (c) The Bank returns the balance information to the ATM.
   (d) The ATM displays the balance to the user.
   (e) The user requests a printed receipt (optional).
      i. If the user requests a receipt, the ATM instructs the Printer to print the balance details, and the receipt is dispensed to the user.
5. The user ends the session:
   (a) The ATM returns the card to the user.

This sequence diagram demonstrates the interactions and method calls between the User, ATM, Bank, and Printer during a balance inquiry transaction.

In the next lesson, we will create activity diagrams for the ATM.

6 Activity Diagram for the ATM System

An activity diagram is a great way to visualize the flow of messages from one activity to another in the system. There can be different activity diagrams that we can create for our ATM. In this lesson, we will create activity diagrams for the following two activities:

- The cardholder performs ATM transactions.
- **Activity challenge:** The cardholder performs an ATM cash withdrawal.

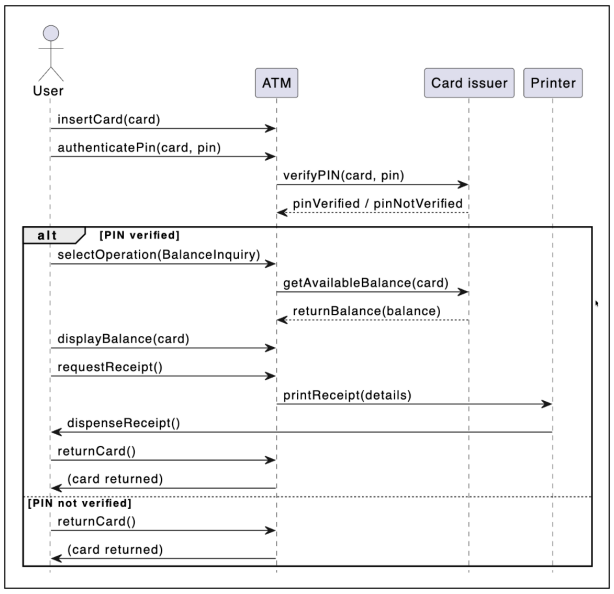


Figure 1.25: The sequence diagram for how to complete a balance inquiry from an ATM

6.1 The cardholder performs ATM transactions

The following are the states and actions that will be involved in this activity diagram.

States

Initial state: The cardholder inserts an ATM card.
Final state: There are two final states in this activity diagram, as shown below:

- The cardholder gets an error message for the wrong ATM PIN.
- The cardholder performs a transaction successfully.

Actions

The cardholder approaches the ATM, inserts their card, and enters their PIN. If the PIN is correct, they are allowed to perform a transaction. If the PIN is incorrect, the ATM returns the card. The cardholder then has the option to select from four different transactions. After completing their chosen transaction, the cardholder receives their card back from the ATM. Based on the order above, the activity diagram is given below.

6.2 Activity challenge: The cardholder performs an ATM cash withdrawal

You'll help us create an activity diagram of a cardholder performing an ATM cash withdrawal. The skeleton of the activity diagram is shown below.

Notice that the actions in the diagram above are numbered from 1 to 6. The slots below represent the activities, and the arrows represent the flow from one activity to another. Can

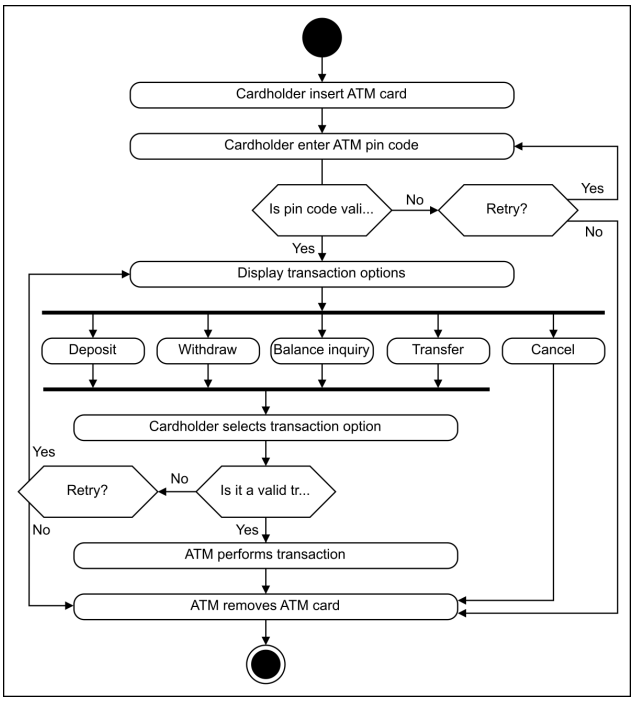


Figure 1.26: The activity diagram for the cardholder performing ATM transactions

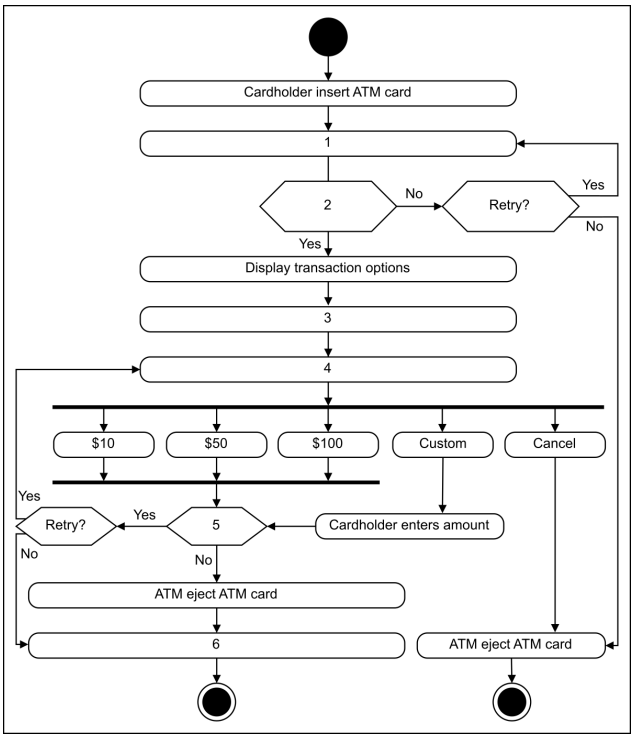


Figure 1.27: The activity diagram of the cardholder performing an ATM cash withdrawal

you rearrange the slots below in the correct order they should appear in the activity diagram above?

> **Note:** If you're stuck, click the "Show Solution" button.

Alternatively, click the "Show complete diagram" button below to see the complete sequence diagram:

We've looked at some of our ATM's activity diagrams. In the next lesson, we will present the code for our designed classes in some of the most popular languages.

7 Code for the ATM System

Using various UML diagrams, we've covered different aspects of the ATM and observed the attributes attached to the problem. Let 's now explore the more practical side of things, where we will work on implementing the ATM using multiple languages. This is usually the last step in an object-oriented design interview process.

We have chosen the following languages to write the skeleton code of the different classes present in the ATM system:

- Java
- C#
- Python
- C++
- JavaScript

If you want to skip directly to the implementation and see the complete workflow, simply scroll down or click Executable Code: ATM.

7.1 ATM classes

This section will provide the skeleton code of the classes designed in the class diagram lesson.

> **Note:** For simplicity, we are not defining getter and setter functions. The reader can assume that all class attributes are private, accessed through their respective public getter methods, and modified only through their public method functions.

Enumerations

The following code defines the enumeration used in the ATM system.

`ATMStatus`: This enumeration keeps track of the following states of an ATM:

- Idle

- Card inserted by the user
- Option selected
- Cash withdrawal
- Money transfer
- Display the account balance

Note: JavaScript does not support enumerations, so we will use the `Object.freeze()` method as an alternative that freezes an object and prevents further modifications.

Definition of enums

```
enum ATMStatus {
    Idle,
    HasCard,
    SelectionOption,
    Withdraw,
    TransferMoney,
    BalanceInquiry
}

enum TransactionType {
    BalanceInquiry,
    CashWithdrawal,
    FundsTransfer,
    ChangePIN,
    Cancel
}
```

User and ATM card

The `User` class stores the user's `ATMcard` and bank account, where the `ATMCard` class holds the card number, customer name, card expiration date, and PIN. The definitions of these classes are provided below:

The User and ATMCard classes

```
public class User {
  private ATMCard card;
  private BankAccount account;
}

public class ATMCard {
  private String cardNumber;
  private String customerName;
  private Date cardExpiryDate;
```

```
  private int pin;
}
```

Bank and bank account

The `Bank` class represents a bank having a name and code. The `BankAccount` class represents a bank account with two child classes: `SavingAccount` and `CurrentAccount`. These derived classes provide a method for getting the withdrawal limit. The definitions of these classes are provided below:

The Bank and bank account classes

```
public class Bank {
    private String name;
    private String bankCode;

    public String getBankCode() {
        return bankCode;
    }
}

public abstract class BankAccount {
    protected int accountNumber;
    protected double totalBalance;
    protected double availableBalance;

    public double getAvailableBalance() {
        return availableBalance;
    }

    public boolean withdraw(double amount) {
        if (amount <= availableBalance) {
            availableBalance -= amount;
            return true;
        }
        return false;
    }

    public boolean transfer(BankAccount toAccount, double amount) {
        if (amount <= availableBalance) {
            availableBalance -= amount;
            toAccount.availableBalance += amount;
            return true;
        }
        return false;
    }

    public abstract double getWithdrawLimit();
}

public class SavingAccount extends BankAccount {
    @Override
```

```
    public double getWithdrawLimit() {
        // Example: return some limit, e.g., 1000
        return 1000.0;
    }
}

public class CurrentAccount extends BankAccount {
    @Override
    public double getWithdrawLimit() {
        // Example: return some limit, e.g., 5000
        return 5000.0;
    }
}
```

Card reader, card dispenser, printer, screen, and keypad

The `CardReader`, `CashDispenser`, `Keypad`, `Screen`, and `Printer` classes compose the ATM and have the following functionalities:

- `CardReader`: Reads the card inserted by the user.
- `CashDispenser`: Dispenses cash upon withdrawal request.
- `Keypad`: Used by the user to enter the PIN for authentication and to input amounts or a new PIN.
- `Screen`: Displays messages.
- `Printer`: Prints receipts.

The definitions of these classes are provided below:

Card reader and other classes

```
public class CardReader {
    public boolean readCard(ATMCard card) {
        // Simulate card reading logic
        return card != null;
    }
}

public class CashDispenser {
    public boolean dispenseCash(int amount) {
        // Simulate dispensing cash
        return true;
    }
}

public class Keypad {
    public String getInput() {
        // Simulate input
```

```
        return "";
    }
}

public class Screen {
    public void showMessage(String message) {
        System.out.println(message);
    }
}

public class Printer {
    public void printReceipt(String details) {
        System.out.println("Receipt: " + details);
    }
}
```

ATM state

`ATMState` is an abstract class extended by the following concrete classes:

- `IdleState` (handles card insertion)
- `HasCardState` (handles PIN authentication)
- `SelectionOptionState` (handles transaction selection)
- `BalanceInquiryState` (handles displaying balance)
- `CashWithdrawalState` (handles cash withdrawal)
- `TransferMoneyState` (handles funds transfer)
- `ChangePinState` (handles PIN changes)

These derived classes derived classes override the methods relevant to their respective states, including `returnCard()` and `exit()`.
The definitions of these classes are provided below:

The ATMState classes

```
public abstract class ATMState {
    public void insertCard(ATM atm, ATMCard card) {}
    public void authenticatePin(ATM atm, ATMCard card, int pin) {}
    public void selectOperation(ATM atm, TransactionType tType) {}
    public void cashWithdrawal(ATM atm, ATMCard card, double amount) {}
    public void displayBalance(ATM atm, ATMCard card) {}
    public void transferMoney(ATM atm, ATMCard card, BankAccount toAccount, double
        amount) {}
    public void changePin(ATM atm, ATMCard card, int newPin) {}
    public void cancelTransaction(ATM atm) {}
    public void returnCard(ATM atm) {}
```

```
    public void exit(ATM atm) {}
}

// Concrete State classes follow this pattern. You can expand their bodies as
    needed.
public class IdleState extends ATMState {}
public class HasCardState extends ATMState {}
public class SelectionOptionState extends ATMState {}
public class CashWithdrawalState extends ATMState {}
public class TransferMoneyState extends ATMState {}
public class BalanceInquiryState extends ATMState {}
public class ChangePinState extends ATMState {}
```

ATM

An `ATM` maintains the following at any given moment:

- A specific operational state, such as `Idle`, `HasCard`, `SelectionOption`, or an active transaction state (e.g., `CashWithdrawal`, `FundsTransfer`, `BalanceInquiry`, `ChangePIN`)
- Current cash balance
- A limited number of bill denominations: one-hundred-dollar, fifty, and ten-dollar bills

It also references all essential hardware components (card reader, cash dispenser, keypad, screen, and printer) and delegates user interactions and transaction processing according to its current state.
The definitions of these classes are provided below:

The ATM class

```
public class ATM {
    private static ATM atmObject = new ATM();
    private ATMState currentATMState;
    private ATMStatus atmStatus;
    private int atmBalance;
    private int noOfHundredDollarBills;
    private int noOfFiftyDollarBills;
    private int noOfTenDollarBills;

    private CardReader cardReader;
    private CashDispenser cashDispenser;
    private Keypad keypad;
    private Screen screen;
    private Printer printer;

    private ATM() {
        // Initialize hardware components
    }

    public static ATM getInstance() {
```

```
        return atmObject;
    }

    public void displayCurrentState() {
        System.out.println("ATM Status: " + atmStatus);
    }

    public void initializeATM(int atmBalance, int noOfHundredDollarBills, int
        noOfFiftyDollarBills, int noOfTenDollarBills) {
        // Initialize balances and cash
    }

    // Delegate to current state
    // ...
}
```

7.2 Executable code: ATM

Below is a fully self-contained, runnable program in Java, C#, C++, Python, and JavaScript that demonstrates the ATM system's core workflows. The system's main driver code resides in `Driver.java`, `Driver.cs`, `Driver.py`, `Driver.cpp`, and `Driver.js` for all respective languages. You can click the "Run" button to execute the codes.

What does this code show

- **System initialization:** Sets up the ATM, users, accounts, cards, and hardware components.
- **Scenario 1 (Cardholder checks account balance):** The user inserts their card, enters their PIN, and performs a balance inquiry.
- **Scenario 2 (Cash withdrawal):** The user withdraws cash, and the system dispenses money according to available denominations.
- **Scenario 3 (Funds transfer):** The user transfers money between accounts and receives a printed receipt.
- **Scenario 4 (Change PIN):** The user changes their ATM card PIN.
- **Scenario 5 (Cancel transaction):** The user cancels a transaction after authentication, and the card is returned.

Hint: Try customizing the code!
This code is designed for experimentation, not just reading. You can edit, extend, or modify any part of the example.

Executable code: ATM

```
import java.util.*;
```

```java
public class Driver {
    public static void main(String[] args) {
        // Set up bank, accounts, users, cards
        Bank bank = new Bank("Sample Bank", "SB001");

        BankAccount acc1 = new SavingAccount(1001, 1200.0);
        BankAccount acc2 = new CurrentAccount(1002, 8000.0);
        ATMCard card1 = new ATMCard("123456", "Alice", "12/27", 1111);
        ATMCard card2 = new ATMCard("654321", "Bob", "08/26", 2222);

        User user1 = new User(card1, acc1);
        User user2 = new User(card2, acc2);

        // Set up ATM
        ATM atm = ATM.getInstance();
        atm.initializeATM(20000, 100, 40, 50); // $20,000, 100x$100, 40x$50, 50x$10

        System.out.println("=== Scenario 1: Alice - Balance Inquiry ===");
        atm.setActiveUser(user1);
        atm.getCurrentATMState().insertCard(atm, user1.getCard());
        atm.getCurrentATMState().authenticatePin(atm, user1.getCard(), 1111);
        atm.getCurrentATMState().selectOperation(atm, TransactionType.BalanceInquiry
            );
        atm.getCurrentATMState().displayBalance(atm, user1.getCard());
        atm.getCurrentATMState().returnCard(atm);

        System.out.println("
=== Scenario 2: Alice - Withdraw $500 ===");
        atm.setActiveUser(user1);
        atm.getCurrentATMState().insertCard(atm, user1.getCard());
        atm.getCurrentATMState().authenticatePin(atm, user1.getCard(), 1111);
        atm.getCurrentATMState().selectOperation(atm, TransactionType.CashWithdrawal
            );
        atm.getCurrentATMState().cashWithdrawal(atm, user1.getCard(), 500.0);
        atm.getCurrentATMState().returnCard(atm);

        System.out.println("
=== Scenario 3: Bob - Transfer $1000 to Alice ===");
        atm.setActiveUser(user2);
        atm.getCurrentATMState().insertCard(atm, user2.getCard());
        atm.getCurrentATMState().authenticatePin(atm, user2.getCard(), 2222);
        atm.getCurrentATMState().selectOperation(atm, TransactionType.FundsTransfer)
            ;
        atm.getCurrentATMState().transferMoney(atm, user2.getCard(), acc1, 1000.0);
        atm.getCurrentATMState().returnCard(atm);

        System.out.println("
=== Scenario 4: Bob - Change PIN ===");
        atm.setActiveUser(user2);
        atm.getCurrentATMState().insertCard(atm, user2.getCard());
        atm.getCurrentATMState().authenticatePin(atm, user2.getCard(), 2222);
        atm.getCurrentATMState().selectOperation(atm, TransactionType.ChangePIN);
        atm.getCurrentATMState().changePin(atm, user2.getCard(), 9999); // Bob sets
            new PIN to 9999
        atm.getCurrentATMState().returnCard(atm);
```

```
        System.out.println("
=== Scenario 5: Alice - Cancel transaction after PIN ===");
        atm.setActiveUser(user1);
        atm.getCurrentATMState().insertCard(atm, user1.getCard());
        atm.getCurrentATMState().authenticatePin(atm, user1.getCard(), 1111);
        atm.getCurrentATMState().selectOperation(atm, TransactionType.Cancel);
        // Card returned, session ends
    }
}
```

7.3 Wrapping up

In this chapter, we've explored the complete design of the ATM, including how a basic ATM system can be visualized using various UML diagrams and designed using object-oriented principles and design patterns.

8 Summary: Designing ATM

Now that you've completed the ATM system case study, let's take a moment to reflect on and consolidate what we've learned. We 'll revisit the key system requirements, identify the core classes along with their responsibilities and relationships, and highlight the major design principles applied. We 'll also examine how objects interact within the system and walk through the overall workflow to understand how the components come together to achieve the desired functionality.

8.1 Key requirements

The following are the primary functional and operational requirements for the ATM system:

1. Each user has a single bank account accessible via an ATM card.
2. Core ATM components include a card reader, a keypad, a screen, a printer, a cash dispenser, and a network interface.
3. ATM must authenticate users using a PIN before permitting transaction access.
4. Only authenticated users can perform transactions.
5. Users may have a current and/or savings account.
6. Supported operations: balance inquiry, cash withdrawal, and fund transfer.
7. After a transaction, users can choose to perform another or end the session and retrieve their card.

8.2 System actors

The ATM system involves a single primary actor:

- `Cardholder:` Interacts with the ATM to perform various banking transactions, including inserting/removing the card, entering the PIN, and initiating actions such as balance inquiry, fund transfer, or cash withdrawal.

8.3 Important classes and relationships

This system includes several interacting classes to handle authentication, transactions, and hardware interfaces.

Class	**Description**	**Important Relationships**
User	Represents a bank customer.	Associated with ATMCard and BankAccount.
ATMCard	Holds card data (number, name, expiry, PIN).	Associated with BankAccount.
BankAccount	Base class for accounts; includes balance and account details.	Inherited by SavingAccount and CurrentAccount.
Bank	Represents the issuing bank.	Associated with ATM.
ATM	Central unit handling transactions, user input/output, and session control.	Associated with Bank, ATMCard, and ATMState; composed of core hardware interfaces.
ATMState	Abstract class for ATM behavioral states.	Extended by CheckBalanceState, CashWithdrawalState, etc.
ATMStatus (Enum)	Represents ATM states (idle, card inserted, etc.).	Used by ATM.
TransactionType (Enum)	Represents transaction categories: BalanceInquiry, CashWithdrawal, FundsTransfer.	Used for transaction classification and UI routing.

Table 1.1: ATM System Classes and Their Relationships

8.4 Design highlights

The ATM design integrates hardware and software elements to create a secure, user-friendly banking terminal.

Design approach

The system was designed using a bottom-up approach, starting from low-level hardware components like `CardReader` and `Keypad`, and building toward complete transaction workflows using `ATMState` abstractions.

Design principles

The system adheres to SOLID principles for robust and maintainable architecture:

- **Single Responsibility principle (SRP):** Each class serves a specific purpose (e.g., `CardReader` reads cards, `CashDispenser` handles cash).
- **Open/Closed principle (OCP):** ATM states and withdrawal logic can be extended without modifying base classes.
- **Liskov Substitution principle (LSP):** Subtypes (e.g., `SavingAccount`) can replace their parent `BankAccount` without breaking functionality.
- **Interface Segregation principle (ISP):** Interfaces like `ATMState` segregate behavior into distinct contexts (e.g., balance checking, withdrawal).
- **Dependency Inversion principle (DIP):** High-level logic in `ATM` depends on abstractions like `ATMState`.

Design patterns

Several classic design patterns are utilized:

- **Singleton pattern:** Ensures only one instance of the `ATM` is active at any given time.
- **State pattern:** Enables the ATM to change behavior based on internal state transitions (e.g., from idle to withdrawal).
- **Chain of Responsibility pattern:** The cash for $100, $50, and $2 bills is distributed via a chain of handlers.
- **Composite and Builder patterns:** Suggested for future extensibility across transaction types and UI flows.

8.5 Object interactions

These key interactions demonstrate how components coordinate to perform ATM operations:

- **Balance inquiry:**
  - User inserts card → enters PIN → system verifies PIN via bank.
  - User selects balance inquiry → ATM requests balance from issuer.
  - Balance is displayed; receipt optionally printed → card returned.
- **Cash withdrawal:**
  - User selects withdrawal → enters amount.

  - System checks ATM cash limits and account limits.
  - Uses `CashDispenser` chain to dispense correct denominations.
  - Receipt printed; card returned.

- **Fund transfer:**
  - Similar to withdrawal, but no cash is dispensed.
  - ATM verifies account balances, updates them accordingly, and prints a receipt.

- **PIN validation and retry:**
  - The ATM card is immediately ejected with an error message if the PIN is invalid.

8.6 System workflow

The workflows below reflect key user scenarios and are modeled after the system's activity diagrams:

- **General ATM transaction:**
  - User inserts card and enters PIN.
  - If the PIN is invalid, → the card is ejected, session ends.
  - If the PIN is valid, → the transaction menu is displayed.
  - User selects transaction → ATM transitions to appropriate state.
  - Transaction executed → receipt offered → card returned.

- **Cash withdrawal:**
  - User selects withdrawal → inputs amount.
  - ATM validates funds and denomination availability.
  - Cash is dispensed using withdrawal processors.
  - Receipt printed → card returned.

- **Balance inquiry:**
  - Follows the same steps as a general transaction, leading to a balance display.

- **Cancel transaction:**
  - At any point, the user can cancel → ATM halts the transaction, returns the card.

9 Key Insights and Tips: Designing ATM

Congratulations on completing the case study for designing an ATM system. This lesson builds on your learning by highlighting common design mistakes, offering key interview tips, and including a short quiz to assess your understanding. You 'll also find related case studies that strengthen your grasp of modular, state-driven object-oriented design.

9.1 Common mistakes

Avoiding these common mistakes will help you create a more reliable and scalable ATM:

- **Poor state modeling:** Using conditionals instead of concrete states like `IdleState`, `HasCardState`, or `CashWithdrawalState` leads to tangled logic. Apply the State pattern to define ATM behavior cleanly and modularly.
- **Hardcoding transaction logic:** Embedding transaction operations like withdrawal or transfer directly inside the ATM class restricts flexibility. Use the Command or Strategy pattern to decouple logic and support future operations.
- **Overlooking card authentication flow:** Skipping steps like card verification, PIN validation, and bank confirmation creates security gaps. Model each phase of authentication as a separate responsibility using state transitions.
- **No withdrawal breakdown logic:** Failing to split withdrawal amounts into correct denominations (e.g., $100, $20, $2) results in unrealistic cash dispensing. Use the Chain of Responsibility pattern to handle each denomination sequentially.
- **Ignoring invalid transaction handling:** Allowing withdrawals that exceed ATM cash or account limits can crash the system. Always validate constraints on both user balance and ATM capacity before processing a transaction.

9.2 Key interview tips

Understand key design patterns, system components, and real-world applications to prepare for your interview and confidently discuss your approach.

9.3 Test your understanding

Now that you understand OOD principles well, solve the quiz below to test your knowledge.

Designing ATM

Question 1: Which design decision best promotes scalability in ATM operations like withdrawals, deposits, and transfers?

Tip	Explanation
Design with extensibility in mind.	The system can support future features like deposits, bill payments, and biometric authentication by keeping class roles flexible and decoupling components.
Separate hardware components.	Compose the ATM with modular components such as Printer, CardReader, and CashDispenser.
Handle cash denominations via the Chain of Responsibility pattern.	Use this pattern to break withdrawal amounts into supported denominations (e.g., $100, $20, $2) efficiently and flexibly.
Abstract bank account types.	Use inheritance to create SavingAccount and CurrentAccount from a BankAccount superclass.
Design for authentication flow.	Ensure card insertion, PIN entry, and bank verification are properly sequenced and modeled.

Table 1.2: Tips for Designing an Extensible ATM System

- Implementing all logic in the ATM class.
- Using a `switch` statement for each transaction type.
- **[CORRECT]** Applying the Strategy or Command pattern for transactions.
  Explanation: Using the Strategy or Command pattern lets you encapsulate each transaction as a separate class, making it easy to add, remove, or modify operations without changing core ATM logic.
- Storing transaction logic inside the `BankAccount` class.

Question 2: How is the cash breakdown logic (e.g., for dispensing $548) handled in a scalable design?

- Nested if-else statements in the ATM class
- Hardcoded loops per bill type
- **[CORRECT]** Chain of Responsibility pattern with processor classes
  Explanation: The Chain of Responsibility pattern enables clean and extensible logic to dispense mixed denominations.
- Delegate to Printer class

Question 3: Why is `ATMState` modeled as an abstract class?

- To provide shared data to members

- To avoid interface implementation
- To allow the reuse of the `Printer` class
- **[CORRECT]** To define common behavior while enforcing specific operation handling in derived classes

  Explanation: `ATMState` is abstract to define a contract while letting derived states handle logic like PIN check or withdrawal.

Question 4: What represents a composition relationship?

- `ATM` has an association with the `Bank`.
- **[CORRECT]** `ATMRoom` owns an ATM.

  Explanation: `ATMRoom` owns the ATM and ensures its lifecycle—this is a composition relationship.
- `ATMCard` links to `BankAccount`.
- `BankAccount` links to `User`.

Question 5: What is the primary reason for using the Chain of Responsibility pattern in ATM cash dispensing?

- To support multiple user roles
- To enable parallel transaction handling
- **[CORRECT]** To break down withdrawal amounts into available denominations cleanly

  Explanation: The Chain of Responsibility pattern allows the ATM to delegate cash dispensing across denomination handlers (e.g., $100, $20, $2), ensuring a clean and scalable way to issue cash.
- To store user PINs securely

9.4 Related case studies

To further strengthen your understanding, explore these related OOD case studies:

- **Vending machine system:** Like an ATM, this involves hardware interaction, payment validation, and state transitions. It commonly uses patterns like State and Strategy to manage item dispensing and handle different payment flows.
- **Online banking portal:** Shares core flows with the ATM, such as authentication, balance inquiry, and fund transfers. At the same time, it's UI-driven, its backend logic and account operations closely parallel ATM transactions.

- **Designing a hotel management system:** This involves check-in/check-out processes, payment handling, and resource availability,all of which reinforce modeling user interaction and transaction control, similar to ATM session management.
- **Designing the Amazon online shopping system:** Teaches how to model user accounts, secure payments, and order state transitions. The checkout and payment workflows mirror withdrawal and verification steps in ATM systems.
- **Designing a restaurant management system:** This approach emphasizes role-based actions, order/payment processing, and system-triggered state changes—helpful for thinking modularly about ATM features like transaction types and receipts.